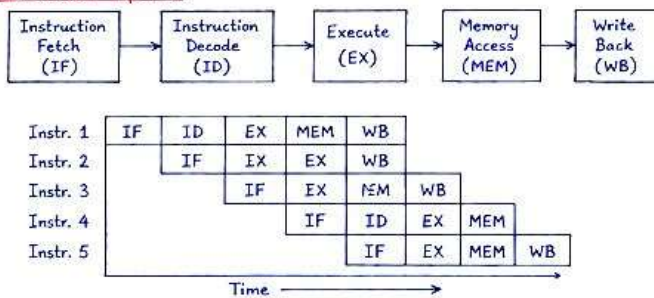
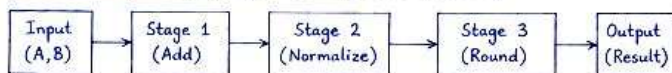


UNIT-1: PIPELINE PROCESSING

## 1. Compare instruction pipeline vs arithmetic pipeline.

| Point                     | Instruction Pipeline                                                     | Arithmetic Pipeline                                                         |
|---------------------------|--------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| 1. Purpose                | Used to increase the throughput of instruction execution.                | Used to speed up arithmetic operations.                                     |
| 2. Operation              | Divides instruction execution into a number of stages.                   | Divides arithmetic operation into sub-operations (stages).                  |
| 3. What flows in pipeline | Different instructions flow through the pipeline.                        | Different parts of the same arithmetic operation flow through the pipeline. |
| 4. Example                | IF → ID → EX → MEM → WB pipeline in CPU.                                 | Floating point addition pipeline.                                           |
| 5. Used in                | CPU (Control Unit + Datapath)                                            | ALU / FPU (Arithmetic Unit)                                                 |
| 6. Aim                    | To improve instruction throughput.                                       | To improve arithmetic processing speed.                                     |
| 7. Parallelism            | Multiple instructions are processed simultaneously, at different stages. | Multiple sub-operations of same instruction are processed simultaneously.   |
| 8. Dependence             | Instructions are independent of each other.                              | Stages are dependent on previous stage result.                              |

Instruction Pipeline:Arithmetic Pipeline (Example: Floating Point Addition)

Instruction pipeline increases instruction throughput while arithmetic pipeline increases arithmetic operation speed.

Page-3

3. What are data hazards in computer architecture?  
How can they be resolved?Answer:

- Data hazard occurs when an instruction depends on the result of a previous instruction that has not completed its execution.

Example: I1: R1 = R2 + R3 → I2 depends on result of I1.  
I2: R4 = R1 + R5

Types of Data Hazards:

1. RAW (Read After Write):
  - The next instruction reads a data before previous instruction writes it.
  - Most common hazard.
2. WAR (Write After Read):
  - A write operation occurs before a previous read operation.
3. WAW (Write After Write):
  - Two instructions write the same location and order becomes wrong.

Pipeline Example (RAW Hazard)

| Clock Cycle →    | 1  | 2  | 3  | 4   | 5  | 6   |
|------------------|----|----|----|-----|----|-----|
| I1: R1 = R2 + R3 | IF | ID | EX | MEM | WB |     |
| I2: R4 = R1 + R5 |    |    | IF | ID  | EX | MEM |

I2 needs R1 in its ID stage but I1 produces R1 in WB stage  
→ Data Hazard (RAW)

## 2. "Instruction execution throughput increases in proportion with the number of pipeline stages." Is it true? Justify.

Answer:

- The statement is approximately true but not always completely true in practical cases.
- In pipeline, instruction execution is divided into a number of stages. More the number of stages, more instructions can be in different stages simultaneously.
- After the pipeline is filled, ideally one instruction is completed in every clock cycle.
- Therefore, throughput increases with number of stages.

$$\text{Throughput} = \frac{\text{Number of Instructions Completed}}{\text{Time Taken}}$$

$$\text{Ideally, Throughput} \propto \text{Number of Pipeline Stages}$$

Example:

- Suppose without pipeline, one instruction takes 20 ns.
- With 5-stage pipeline, after initial delay, one instruction completes every 4 ns.
- So throughput increases.

But in practical:

- Throughput does not increase in exact proportion because of -
  - (i) Pipeline hazards (data, control, structural)
  - (ii) Memory access delay
  - (iii) Branch instructions
  - (iv) Overhead of pipeline registers

Conclusion:

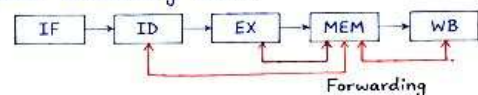
Increasing pipeline stages improves throughput but actual performance depends on hazards, hardware and design.

Page-4

Methods to Resolve Data Hazards:

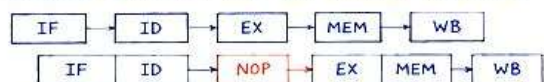
## 1. Operand Forwarding (Bypassing):

- The result is forwarded directly from the output of a later stage to the input of an earlier stage.
- Avoids unnecessary stalls.



## 2. Pipeline Stalling (Insertion of NOP):

- If forwarding is not possible, pipeline is stalled by inserting one or more NOP (No Operation).
- Increases execution time but ensures correctness.



## 3. Instruction Reordering (Scheduling):

- Rearrange independent instructions between dependent instructions.
- Compiler or hardware reorders to avoid hazards.

I1 → I2 becomes I1 → I3 → I2

## 4. Register Renaming:

- Removes false dependencies (WAR, WAW hazards).
- Uses different physical registers for same logical registers.

Advantages of Hazard Resolution:

- Improves CPU performance.
- Increases pipeline efficiency.
- Reduces instruction execution delay.

Proper handling of hazards is essential for high performance pipelined processors.